

Lab 1: Building Your First Intrusion Detector

1. What is this lab about?

An **Intrusion Detection System (IDS)** watches network traffic and raises an alarm when something looks like an attack (a scan, a flood, a break-in attempt). Instead of writing rules by hand, we let a machine-learning model *learn* the difference between normal and malicious traffic from examples. That is exactly what you will build in this lab.

You will do this in two forms:

- **Binary detection** — for each connection, answer a yes/no question: is this *normal* or an *attack*?
- **Multiclass detection** — go further and name *which kind* of attack it is (scan, denial-of-service, etc.).

2. What you will be able to do afterwards

- Load a real cybersecurity dataset and get it ready for machine learning.
- Train several models and let the computer learn to spot attacks.
- Measure how good each model is and understand why “high accuracy” can still be a bad detector.
- Compare the models and say, with evidence, which one you would trust.

3. Tools and setup

- **Language:** Python 3. Work in a Jupyter notebook or google colab or plain scripts, you can use what language you prefer.
- **Core libraries:** pandas (load/clean data), scikit-learn (the classical models and metrics), matplotlib (plots).
- **For the neural network:** Keras/TensorFlow or PyTorch, pick whichever you find friendlier. scikit-learn’s MLPClassifier also counts and is the easiest to start with.
- **Install once:** pip install pandas scikit-learn matplotlib (add tensorflow or torch if you use them).

4. The dataset

Use **CICIDS2017** or **CICIDS2018**, all come from the Canadian Institute for Cybersecurity (CIC). Each row is one network connection described by numeric features, plus a label.

Make life easier

Datasets are big. For a first run, load one day’s file (or a random 10–20% sample). Get everything working on the small set, then scale up if you have time.

Cleaning you must do (and why)

Remove “ID” columns like IP addresses, timestamps, or Flow ID. They act like name tags and let the model cheat instead of learning real attack behaviour.

Fix broken numbers: some speed features contain “infinity” or blank (NaN) values. Replace or drop them, or later steps will crash.

Remove exact duplicate rows and trim stray spaces in the label text before you use it.

5. Which models to use

Pick **one** of the classical models below, and **one** neural network.

Model	What it is / beginner tip
Logistic Regression (LR)	The simplest starting point: draws a boundary between normal and attack. Needs scaled features. Fast.
Decision Tree (DT)	A flowchart of yes/no questions. Very easy to understand and to explain. No scaling needed.
Random Forest (RF)	Many decision trees voting together. Usually strong and forgiving. No scaling needed.
Support Vector Machine (SVM)	Finds the widest gap between the classes. Powerful but can be slow on big data , use a sample. Needs scaled features.
Neural network (choose one)	An MLP (multi-layer perceptron) is the recommended, simplest neural net. You may use any NN you like. Needs scaled features.

Scaling cheat-sheet

LR, SVM and neural networks work much better when features are put on the same scale (standardisation). Decision Tree and Random Forest do not need it. When in doubt, scale.

6. Step-by-step tasks

Step 1: Load and look

- Load the data with pandas. Print the shape, the columns, and how many rows are normal vs. attack. Make one bar chart of the class balance.

Step 2: Clean

- Remove the ID columns, fix infinity/NaN values, drop duplicates

Step 3: Split into train / validation / test

- Use a stratified split (e.g. 60/20/20) so each part keeps the same normal-to-attack ratio. Fix the random seed so your results are repeatable.

Step 4: Scale features (for the models that need it)

- Fit the scaler on the training data only, then apply it to validation and test.

Step 5: Train your classical models (binary task)

- Train your two chosen classical models to tell normal from attack. Try a couple of settings for each and keep the best on the validation set.

Step 6: Train your neural network (binary task)

- Train one neural network on the same task. Start small (e.g. one or two hidden layers)

Step 7: Run one “change one thing” experiment (ablation)

What an ablation is

Change exactly one thing, keep everything else the same, and see what happens to the scores. It shows which choices actually matter.

- Pick one: (a) with vs. without feature scaling
- (b) all features vs. fewer features. Report the before/after numbers.

Step 8: Compare your results:

- Put all models in one results table (accuracy, macro-F1, recall, ROC-AUC, FAR).
- **Right metric:** because the data is imbalanced, judge with macro-F1 and FAR, not accuracy alone.

7. What to hand in

Your code. A notebook or scripts plus a short README or PDF that lists the libraries and how to run it. Include the random seed.

A short report (2–3 pages). Suggested sections:

- What the problem is (IDS, binary vs. multiclass) — a short paragraph.
- What you did (cleaning, split, which models, which settings).
- Results (one clear table + one or two figures, each with a caption you actually refer to).
- Discussion (which model wins, what FAR tells you).
- Who did what (one or two lines).

How your work is graded

Every lab is graded 3 / 4 / 5 / F on the same four things. You do not need to be perfect to pass , you need the work to run, to make sense, and to be clearly reported. The table shows what “good” looks like for this lab.

Criterion	Weight	What good work looks like here
Implementation and code quality	30%	Code runs start-to-finish and reproduces your numbers; a README explains how to run it; seed is fixed; reasonably tidy.
Experimental design and methodology	25%	Stratified train/val/test split; no test-set peeking; at least one classical model and one neural network
Results and analysis	20%	Accuracy, macro-F1, recall, ROC-AUC and FAR report for models’ comparison
Report quality	25%	Clear 2–3 pages; captioned table/figures used in the text; who-did-what line included.

Submitting your work

- **Where.** Upload to CANVAS by the deadline: your code (a zip or a repository link) and the report as a PDF. Late submissions are not accepted in general.
- **Group work.** Work together for real. Add a short line at the end of the report saying who did what. If one person does everything, that shows up in individual grades.
- **Second chance.** If a lab does not reach grade 3 you may resubmit it once, before the final reporting date in the schedule.
- **Honesty.** You may use libraries, tutorials, and public code , just cite them. If you use an AI assistant, say so in the report. Your results must come from your own submitted code.

Common mistakes to avoid

- Reporting only accuracy, always show all metrics.
- Forgetting to scale features for LR, SVM and the neural network (their scores will look mysteriously bad).
- Starting on the full CICIDS2017 file and getting stuck , use a sample first, then scale up.

Where to read more

scikit-learn documentation — “Supervised learning” and “Model evaluation” user guides.

- Sharafaldin, I., Habibi Lashkari, A., & Ghorbani, A. A. (2018, January). A detailed analysis of the cicids2017 data set. In *International conference on information systems security and privacy* (pp. 172-188). Cham: Springer International Publishing.
- CICIDS2017 download from : <https://www.unb.ca/cic/datasets/ids-2017.html> or Kaggle
- CICIDS2018 download from : <https://www.unb.ca/cic/datasets/ids-2018.html> or Kaggle